Time Series forecasting using SARIMAX

Soubhik Khankary · Follow

5 min read · Jan 31, 2022

Hello Everyone, In one of my previous post we discussed about how to forecast a variable using classic time series model(ARIMA). In this post I will be showing you how to use one of the classic time series model(SARIMAX) in case if your dataset has got seasonal trends associated with it. Seems like everyone is excited and let's jump quickly into it. In case if you are not sure about ARIMA model please follow up the post below:

UNIVARIATE VARIABLE TIME SERIES FORECASTING USING ARIMA USING PYTHON

Problem Statement: I was trying to solve one of the problem statement which would help to forecast the univariate...

medium.com

Theory: SARIMAX is a combination of four different modules i.e.

S-> It stands for seasonality. In case if you identify that the data patterns is repeating every month /year then yes it is seasonality.

AR-> Auto Regressive means like my current value is dependent on which all lagged values or past values.

I-> In case if your dataset is not stationary you keep trying to make it stationary in case of ARIMA model right like we did in the post mentioned above. I stands for number of differentiation we create with the previous existing values.

MA-> Moving Average and to extent we need to roll it up.

X-> It stands for other exogeneous variables which causes the variable to change. It is used while we are doing multi variate time series analysis.

Let's start the coding stuff as we are going to make the implementation of classic time series model very easy .

Let's start by importing the very basic modules first:

```
#Please install the below two packages and then start working on the module
#pip install pmdarima
#pip install statsmodels --upgrade
#https://www.kaggle.com/rakannimer/air-passengers
```

Please import the packages as it is required for the practicals

```
[ ]  ####Importing basic modules#########
     import pandas as pd
     import numpy as np
     import seaborn as sns
     import matplotlib.pyplot as plt
     %matplotlib inline
```

Basic modules imported

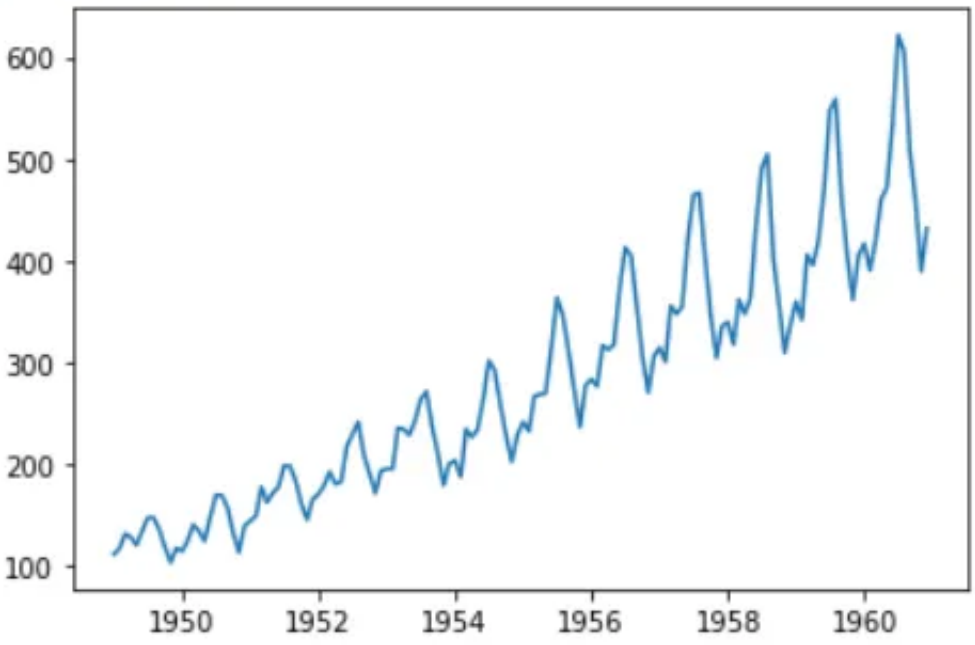
```
[ ]  df1=pd.read_csv("AirPassengers.csv")
     df1['Month']=pd.to_datetime(df1['Month'])
     df1.index=df1['Month']
     df1=df1[['#Passengers']]
     df1.head()
```

Month	#Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Reading the dataset from Kaggle and setting the date column as index

```
[ ]  plt.plot(df1['#Passengers'])
```

```
[<matplotlib.lines.Line2D at 0x7f7315644590>]
```



Visualizing the dataset

I could visualize with my skills of exploratory data analysis that the data has an upward trend, yearly seasonality and small residuals. Let's try to apply some more coding skills.

We need to check and identify whether my dataset is stationary or not. I believe you all know by know how to check. In case if you don't please read the blog above. I have also mentioned in short with the coding snippet.

In this case I am applying ADF test(Augmented Dickey Fuller Test). I will be showing you how to do it in two different ways(**OLD WAY**):

```
''' As discussed earlier we have to check whether our data is stationary in nature or not
Exploratory Data Analysis(Less Reliable ):
    1)Check the rolling mean is constant with time
    2) Check std is constant with time
             or
Augemnted Dickey Fuller Test(More reliable):
    1) Do the hypothesis testing and verify if the value of p <0.05 . In that case hypothesis or assumption fails and data becomes stationary.

'''
from statsmodels.tsa.stattools import adfuller
res=adfuller(df1)
if res[1]>0.05:
  print("The data is not stationary")
else:
  print("The data is stationary")
```

```
The data is not stationary
```

Data is not stationary. fails hypothesis test.

Let's do the same test in other way(**NEW WAY**):

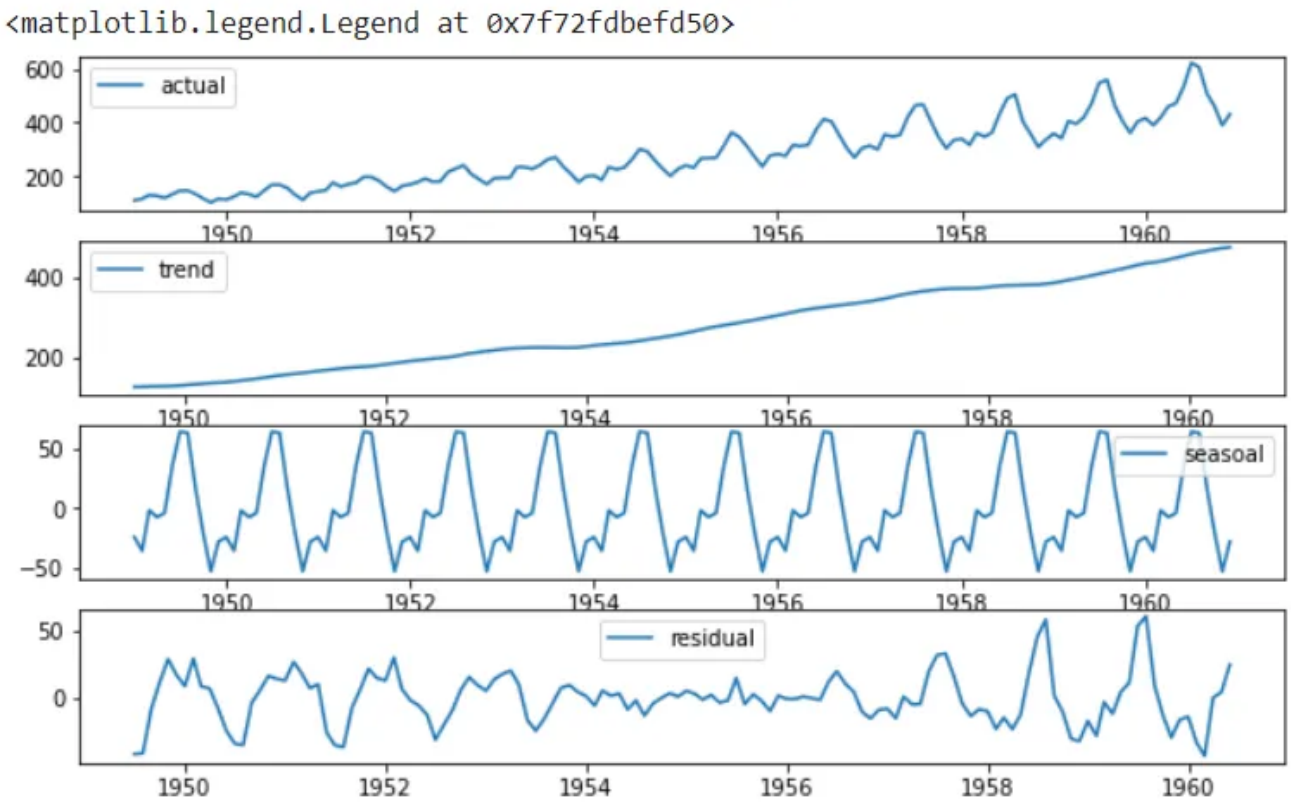
```
'''
Another way of applying the augmented dickey fuller test using pmdarima package
'''
from pmdarima.arima import ADFTest
adftest=ADFTest(alpha=0.05)
res2=adftest.should_diff(df1)
if res2 is False:
  print("Data is not stationary")
else:
  print("Data is stationary")
```

Data is not stationary. fails hypothesis test.

Let's try to visualize the four components of the data which in turn constitute a time series.

```
[ ]  from statsmodels.tsa.seasonal import seasonal_decompose
     decom=seasonal_decompose(df1['#Passengers'])
     dftend=decom.trend
     dfsea=decom.seasonal
     dfres=decom.resid
     plt.figure(figsize=(10,6))
     plt.subplot(411)
     plt.plot(df1,label='actual')
     plt.legend()
     plt.subplot(412)
     plt.plot(dftend,label='trend')
     plt.legend()
     plt.subplot(413)
     plt.plot(dfsea,label='seasoal')
     plt.legend()
     plt.subplot(414)
     plt.plot(dfres,label='residual')
     plt.legend()
```

Code for seasonal_decompose



EDA for 4 components in time series

Now as I have identified my data is not stationary I have to do lot of maths, differentiation and many other stuffs to make the data stationary to apply time series model. Also I have to perform pacf(partial auto co-relation) and acf(auto co-relation) test to determine the values of p and q. Let's try to make this process easier using auto_arima package.

```
[ ]  from pmdarima.arima import auto_arima
     model=auto_arima(df1['#Passengers'],start_p=1, start_q=1,d=1,max_p=5,max_q=5,max_d=5,m=12,start_P=0,D=1,start_Q=0,
                      max_P=5,max_D=5,max_Q=5,seasonal=True,information_criteria='AIC')
```

Just one line solves all our problem

I am commanding my code to try multiple combinations of p, d and q which start from value 1 and goes up to max value of 5. It checks based on AIC value and tries to return me the best values for p, d and q. Seems so simple

and easy.

In short, Lower the value of AIC and in return you get the better model.

```
[ ]  model.summary()
```

SARIMAX Results

Dep. Variable:	y	**No. Observations:**	144
Model:	SARIMAX(0, 1, 1)x(2, 1, [], 12)	**Log Likelihood**	-505.589
Date:	Mon, 31 Jan 2022	**AIC**	1019.178
Time:	08:50:15	**BIC**	1030.679
Sample:	0	**HQIC**	1023.851
	- 144		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ma.L1	-0.3634	0.074	-4.945	0.000	-0.508	-0.219
ar.S.L12	-0.1239	0.090	-1.372	0.170	-0.301	0.053
ar.S.L24	0.1911	0.107	1.783	0.075	-0.019	0.401
sigma2	130.4480	15.527	8.402	0.000	100.016	160.880

Ljung-Box (L1) (Q):	0.01	**Jarque-Bera (JB):**	4.59
Prob(Q):	0.92	**Prob(JB):**	0.10
Heteroskedasticity (H):	2.70	**Skew:**	0.15
Prob(H) (two-sided):	0.00	**Kurtosis:**	3.87

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Computer suggested me to use SARIMAX. I even got the parameters (wowwww) moment!!!

```
[ ]  model.summary()
```

SARIMAX Results

Dep. Variable:	y	**No. Observations:**	144
Model:	SARIMAX(0, 1, 1)x(2, 1, [], 12)	**Log Likelihood**	-505.589
Date:	Mon, 31 Jan 2022	**AIC**	1019.178
Time:	08:50:15	**BIC**	1030.679
Sample:	0	**HQIC**	1023.851
	- 144		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ma.L1	-0.3634	0.074	-4.945	0.000	-0.508	-0.219
ar.S.L12	-0.1239	0.090	-1.372	0.170	-0.301	0.053
ar.S.L24	0.1911	0.107	1.783	0.075	-0.019	0.401
sigma2	130.4480	15.527	8.402	0.000	100.016	160.880

Ljung-Box (L1) (Q):	0.01	**Jarque-Bera (JB):**	4.59
Prob(Q):	0.92	**Prob(JB):**	0.10
Heteroskedasticity (H):	2.70	**Skew:**	0.15
Prob(H) (two-sided):	0.00	**Kurtosis:**	3.87

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Now let's try to split our data into train and test dataset. We have to train our model with Sarimax, test it and then forecast future values. It is our main goal.

Splitting

Checking the look and feel of train and test dataset

Let’s bring in the use of statsmodels package and try to implement SARIMAX model into action.

Let’s predict the results for test dataset. The process is quite interesting and a bit different as it is a time series model.

Let’s concatenate the predicted and actual values and keep it side by side.

seems good

Let’s try to visualize actual, predicted and forecasted values. I got amazing results.

Seems awesome right!!!

Please remember SARIMAX p,d,q values could also be determined by the older way doing pacf and acf test as discussed in earlier posts. I have tried to make the process a bit easier so that even a non-techie guy could also implement and use it. Please let me know in case if you have any doubts or concerns. Any comments would help me to grow more faster towards the data science path.